

Raport projektowy

Generator dróg wspinaczkowych Kilter (Deep Learning)

Autor: Eryk Kopciuch

7 czerwca 2026

Spis treści

1	Opis projektu	3
1.1	Wprowadzenie do ścian wspinaczkowych Kilter	3
1.2	Opis problemu i sformułowanie zadania	3
1.3	Charakterystyka zbioru danych	4
1.4	Warstwy funkcjonalne projektu	6
1.5	Struktura projektu	6
1.6	Zakres implementacyjny projektu	7
2	Wstęp teoretyczny	8
2.1	Sieci neuronowe — pojęcia podstawowe	8
2.2	Tokeny specjalne w modelach sekwencyjnych	9
2.3	Modelowanie sekwencji warunkowych	10
2.4	Formalizacja celu uczenia	10
2.5	Rekurencyjne sieci neuronowe (RNN, LSTM, GRU)	11
2.6	Uczenie przez teacher forcing i funkcja kosztu	13
2.7	Dekodowanie probabilistyczne i wpływ parametrów generacji	13
3	Realizacja sieci	15
3.1	Reprezentacja danych i przygotowanie batchy	15
3.2	Architektura modelu	16
3.3	Procedura uczenia	18
3.4	Optymalizator AdamW	19
3.5	Dobór zbioru walidacyjnego a walidacja krzyżowa	21
3.6	Generacja i postprocessing	22

4	Wyniki eksperymentów	23
4.1	Stanowisko eksperymentalne	23
4.2	Wpływ parametrów uczenia	23
4.3	Wpływ parametrów generacji	29
5	Analiza jakości i ograniczenia	33
5.1	Mocne strony modelu	33
5.2	Obserwowane błędy	34
5.3	Ograniczenia zastosowanej metodyki	34
6	Wnioski	35
7	Dalsze kierunki rozwoju	36

1 Opis projektu

1.1 Wprowadzenie do ścian wspinaczkowych Kilter

Kilter Board jest interaktywną, regulowaną ścianą treningową, której powierzchnia pokryta jest stałym, znormalizowanym układem chwytów podświetlonych diodami LED. Każdy chwyt posiada jednoznaczny identyfikator otworu (*hole id*), a oprogramowanie producenta pozwala użytkownikowi wybrać istniejącą *trasę* (ang. *problem*, *boulder*) z bazy społecznościowej. Po wyborze trasy odpowiednie chwytły zostają zapalone na zielono (chwyt startowy), niebiesko (chwytły pośrednie), fioletowo (chwytły na nogi) i czerwono (chwyt szczytowy). Podobną mechanikę oferują systemy bliźniacze, m.in. *MoonBoard* i *Tension Board*, jednakże niniejszy projekt koncentruje się wyłącznie na układach Kilter ze względu na publicznie dostępny zbiór danych (stfamod, 2024).

Trasę na ścianie definiuje uporządkowany zbiór pozycji chwytów wraz z ich rolami (start, chwyt pośredni, top, chwyt na nogę). Tworzenie nowych tras realizują zwykle eksperci-wspinacze. Automatyczne generowanie tras o zadanej trudności dla wybranego układu panelu jest zatem zadaniem złożonym, łączącym aspekt geometryczny (geometria użytych chwytów), biomechaniczny (możliwości ruchowe wspinacza) oraz dydaktyczny (spójność trudności na całej długości drogi).



Rysunek 1: Fizyczna ściana Kilter Board z systemem chwytów podświetlanych LED.

1.2 Opis problemu i sformułowanie zadania

Projekt dotyczy opracowania warunkowego modelu generatywnego, którego zadaniem jest synteza sekwencji chwytów (tokenów trasy) dla systemu Kilter Board. Problem został sformalizowany jako generacja sekwencji dyskretnej (sekwencja tokenów ze skończonego słownika $\mathcal{V}$) z krótkiego kontekstu warunkującego:

- identyfikatora układu panelu (`layout_id`) — jeden z dostępnych wariantów geometrii ściany,
- identyfikatora trudności (`grade_id`) — ocena V/Font (np. 6a/V3).

Każda próbka ucząca ma postać:

$$\text{source} = [\text{layout_id}, \text{grade_id}], \quad \text{target} = [t_1, t_2, \dots, t_n], \quad (1)$$

gdzie $t_i \in \mathcal{V}$ to i -ty token sekwencji trasy (identyfikator chwytu lub rola), a n to liczba kroków w trasie. Wektor `source` określa warunek generacji, a `target` stanowi wzorzec sekwencji pozycji chwytów.

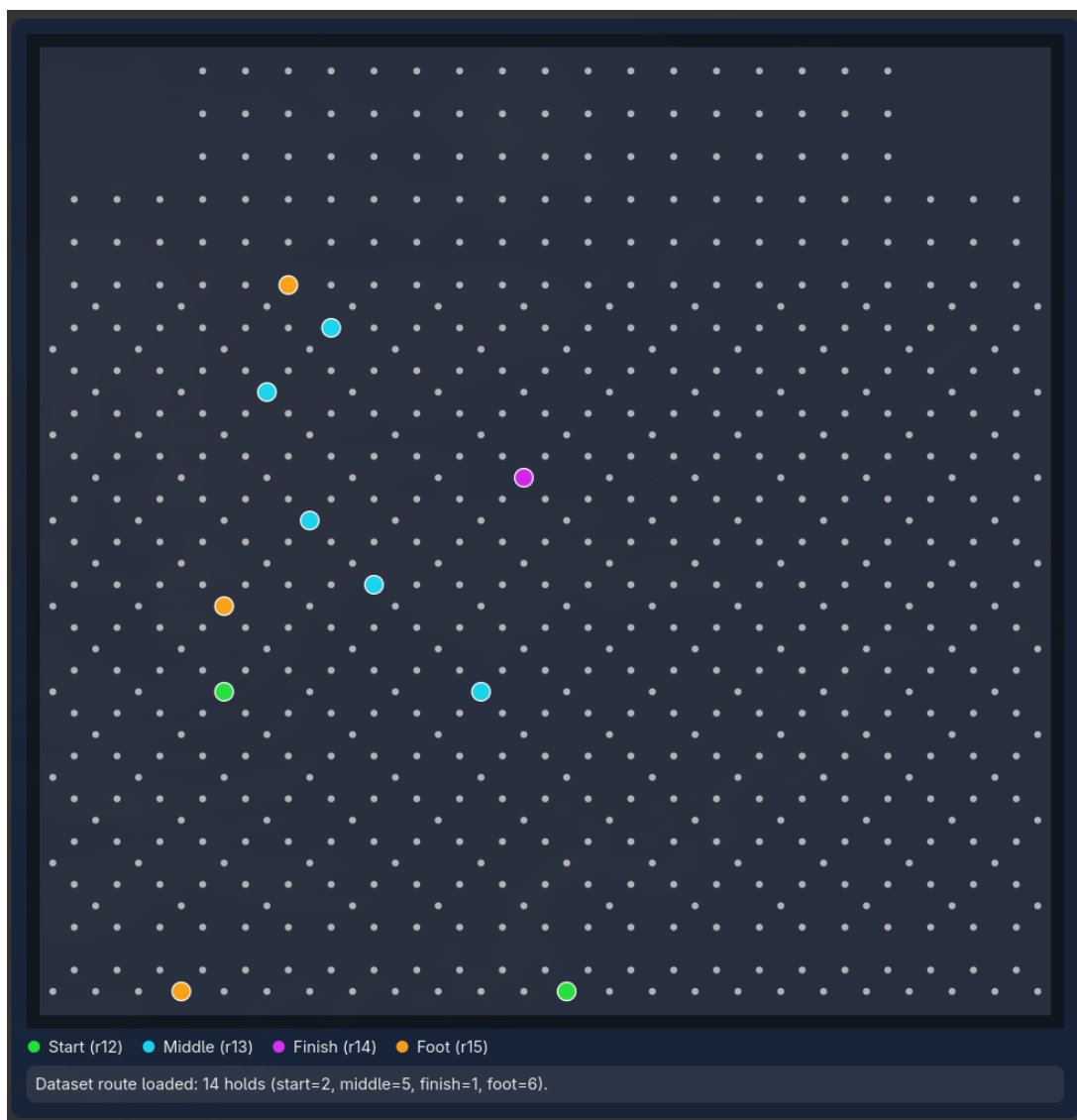
1.3 Charakterystyka zbioru danych

Wykorzystano publicznie dostępny zbiór tras Kilter Board (stfamod, 2024), przefiltrowany według następujących kryteriów:

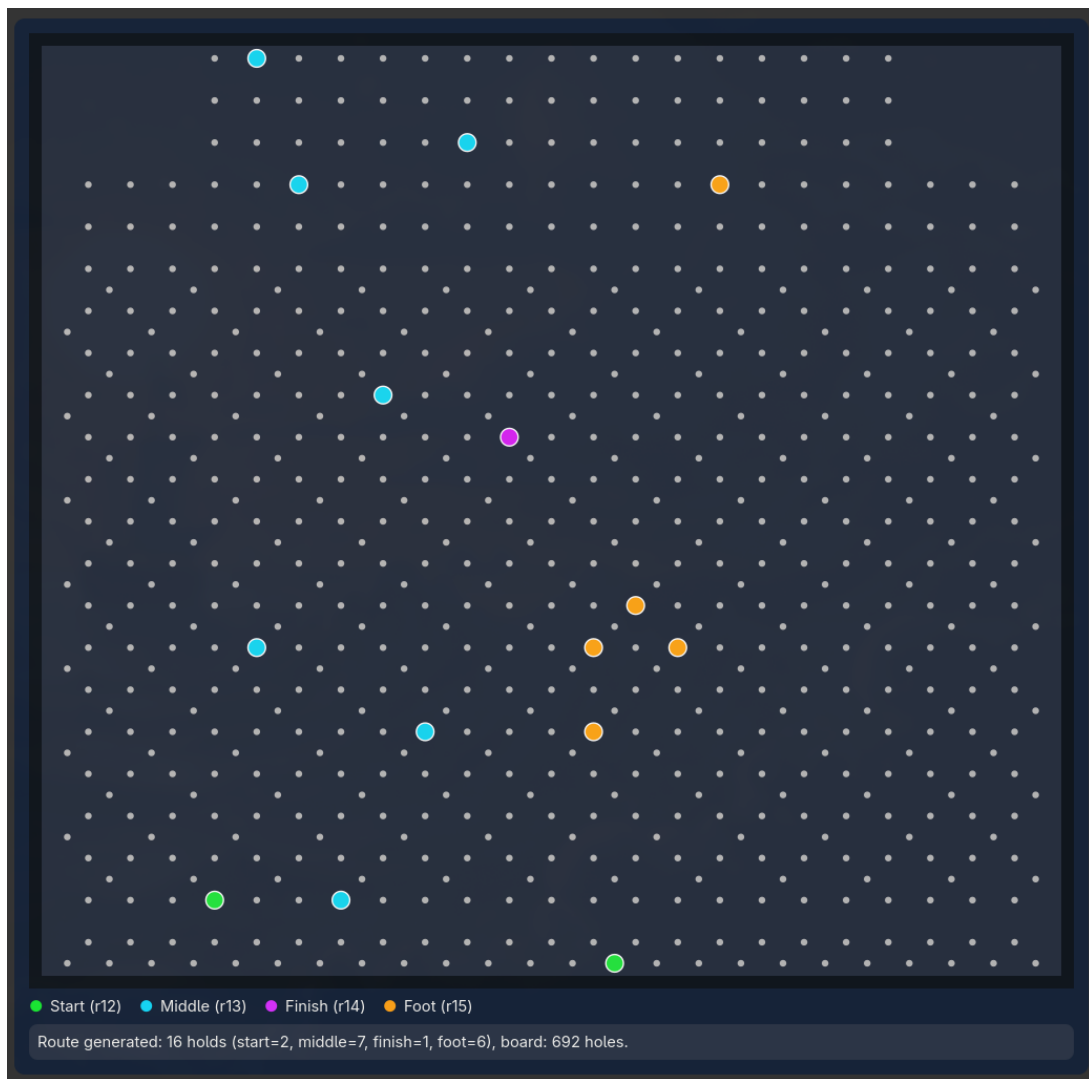
- minimalna liczba pierwszych przejść (*ascensionists*): ≥ 5 ,
- układ: *Kilter Board Original* oraz *Kilter Board Homewall*,
- średnia ocena jakości (*quality rating*): $> 2,6$,
- liczba klatek (*frames count*): 1.

W rezultacie korpus zawiera $N = 222\,624$ tras po wstępnym przetwarzaniu. Dla każdej próbki przygotowano słownik tokenów (`token_to_id`, `id_to_token`) o stałym rozmiarze, w którym wyróżniono trzy klasy tokenów:

- **tokeny pozycji chwytów** (np. p123) — numery otworów panelu,
- **tokeny ról chwytów** (start, top, hold, foot),
- **tokeny specjalne**: <PAD>, <SOS>, <EOS> (sekcja 2.2).



Rysunek 2: Przykładowa trasa z korpusu treningowego, zrenderowana w autorskim GUI.



Rysunek 3: Trasa wygenerowana przez model dla zadanej pary warunkującej.

1.4 Warstwy funkcjonalne projektu

Realizacja projektu składa się z trzech warstw funkcjonalnych:

- **warstwa danych** — wczytanie korpusu tras, mapowanie tokenów oraz podział trening/walidacja,
- **warstwa modelu** — sieć neuronowa typu warunkowy dekodery autoregresyjny (rozdz. 3.2),
- **warstwa inferencji** — generacja tras z kontrolą losowości i przetwarzanie wyników do postaci użytecznej dla integracji z dowolnym interfejsem użytkownika.

1.5 Struktura projektu

Implementacja jest podzielona na trzy moduły, których krótki opis poniżej pozwala dalej w raporcie odwoływać się do nich *nazwami funkcjonalnymi* (np. „moduł treningowy”, „skrypt scenariuszy”) zamiast bezpośrednio do ścieżek plików:

- **moduł sieci** (katalog `kilter_dl/`) — jądro deep-learningowe projektu:
 - *definicja modelu* — klasa `KilterRouteGenerator`, embeddingi warunkowe, deko-der GRU oraz głowa wyjściowa (rozdz. 3.2),
 - *moduł treningowy* — wczytanie danych, grupowanie próbek w batche (funkcja `collate_batch`), pętla uczenia AdamW, zapis checkpointów i metryk epokowych,
 - *moduł inferencji* — generacja autoregresyjna z parametrami τ, k, p, ρ (rozdz. 2.7),
 - *skrypt scenariuszy eksperymentalnych* — uruchamia 11 wariantów grupy A (uczenie) i 14 wariantów grupy B (generacja); wyniki tabelaryzowane w rozdz. 4,
 - *katalog logów eksperymentów* (`logs/exp_A*/`) — metryki epokowe oraz podsumowanie konfiguracji każdego przebiegu uczenia,
 - *katalog checkpointów* (`checkpoints/`) — najlepsze wagi modelu na walidacji, w tym *checkpoint bazowy* (`exp_A0_baseline.pt`) używany do wszystkich scenariuszy generacji,
 - *katalog próbek* (`samples/exp_B*/`) — wygenerowane trasy w formacie JSON.
- **moduł raportowy** (katalog `kilter_report/`) — niezależna biblioteka Pythona do produkcji wykresów raportu z surowych logów i plików próbek:
 - *generator wykresów* — renderuje wszystkie figury sekcji 4 do plików PDF (wekto-rowo),
 - *skrypt uruchomieniowy* — pojedynczy punkt wejścia do procesu generowania ry-sunków.
- **moduł raportu** (katalog `report/`) — źródła niniejszego dokumentu LaTeX (`raport.tex`, `raport.bib`) wraz z podkatalogiem `figures/` na wyrenderowane wykresy.

W dalszej części raportu odniesienia do kodu i artefaktów eksperymentów przyjmują po-stać krótkich nazw funkcjonalnych zdefiniowanych powyżej.

1.6 Zakres implementacyjny projektu

Poniższy fragment kodu ilustruje główne etapy procesu uczenia:

```
token_to_id = load_json(args.token_map)
pad_id = token_to_id["<PAD>"]
sos_id = token_to_id["<SOS>"]
eos_id = token_to_id["<EOS>"]

samples_raw = load_json(args.dataset)
layout_to_idx, grade_to_idx = build_layout_grade_ids(samples_raw)
```

```

samples = [remap_source(s, layout_to_idx, grade_to_idx) for s in samples_raw]

model = KilterRouteGenerator(...).to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=args.lr)

```

Listing 1: Szkielet procedury treningowej — moduł treningowy.

Powyższy kod formalizuje trzy kluczowe decyzje projektowe: (i) jawne użycie tokenów specjalnych <PAD>, <SOS>, <EOS> (sekcja 2.2), (ii) separację przestrzeni tokenów wejściowych warunku od indeksacji używanej przez embeddingi warunkowe, (iii) optymalizację parametryczną metodą AdamW (Loshchilov and Hutter, 2019).

Dla konfiguracji eksperymentalnej oznaczonej jako **report_full** zastosowano pełny zbiór danych:

- liczba próbek całkowita: 222 624,
- zbiór treningowy: 220 397,
- zbiór walidacyjny: 2 227,
- najlepszy wynik walidacyjny: `best_val_nll = 0.8576`.

Powyższe wartości wskazują, że model osiąga użyteczny poziom dopasowania do rozkładu danych treningowych przy zachowaniu zadowalającej jakości na walidacji. Szersza dyskusja schematu doboru zbioru walidacyjnego znajduje się w sekcji 3.5.

2 Wstęp teoretyczny

2.1 Sieci neuronowe — pojęcia podstawowe

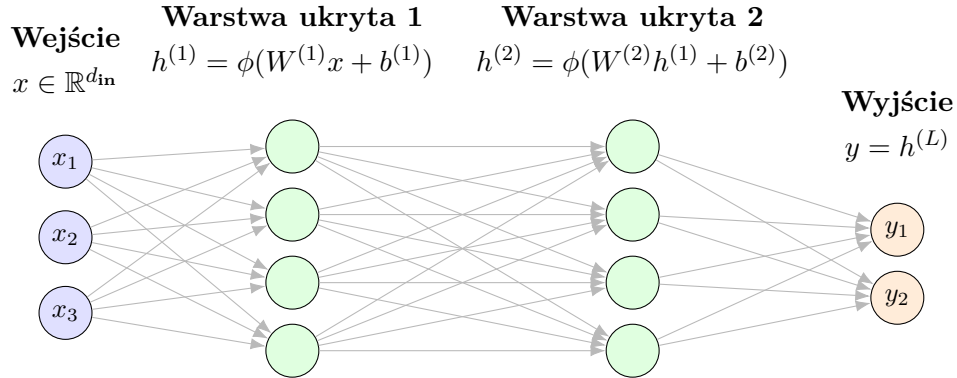
Zanim przejdziemy do specyficznych zagadnień modelowania sekwencji, warto przypomnieć ogólny schemat działania sieci neuronowej. *Sieć neuronowa* to parametryczna funkcja $f_\theta : \mathbb{R}^{d_{\text{in}}} \rightarrow \mathbb{R}^{d_{\text{out}}}$ zbudowana ze złożonych transformacji afinicznych przeplatanych nieliniowościami:

$$h^{(\ell)} = \phi^{(\ell)}(W^{(\ell)}h^{(\ell-1)} + b^{(\ell)}), \quad \ell = 1, \dots, L, \quad (2)$$

gdzie:

- $h^{(0)} = x \in \mathbb{R}^{d_{\text{in}}}$ — wejście,
- $W^{(\ell)}, b^{(\ell)}$ — wagi i bias warstwy ℓ ,
- $\phi^{(\ell)}$ — funkcja aktywacji (np. tanh, ReLU, sigmoidalna),
- $h^{(L)}$ — wyjście modelu.

Kluczową cechą sieci jest *uczenie* parametrów $\theta = \{W^{(\ell)}, b^{(\ell)}\}$ poprzez minimalizację funkcji kosztu $\mathcal{L}(\theta)$ metodą gradientu wstecznego (*backpropagation*) (Rumelhart et al., 1986). Schemat poglądowy gęstej sieci feedforward przedstawia rysunek 4.



Rysunek 4: Wektorowy schemat gęstej sieci neuronowej feedforward. Każdy węzeł reprezentuje jednostkę obliczeniową, każda krawędź — wagę $W_{ij}^{(\ell)}$. Sygnał przepływa od wejścia x przez kolejne warstwy ukryte $h^{(\ell)}$ aż do wyjścia y . Analogiczną zasadę przepływu wykorzystuje dekodery GRU zastosowany w projekcie (rys. 5), z tą różnicą, że kolejne transformacje są rozwijane w czasie po krokach sekwencji, a parametry są współdzielone między krokami.

W projekcie wykorzystano specjalizowaną odmianę takiej sieci — *rekurencyjną sieć neuronową* (rozdz. 2.5) — w której kolejne transformacje powstają przez *rozwiniecie w czasie*: ten sam zestaw wag jest aplikowany w każdym kroku sekwencji, a stan ukryty h_t pełni rolę reprezentacji historii przenoszonej między krokami.

2.2 Tokeny specjalne w modelach sekwencyjnych

Modele sekwencyjne typu encoder–decoder oraz dekodery autoregresyjne operują na słownikach tokenów $\mathcal{V}$ zawierających zarówno tokeny *semantyczne* (reprezentujące element domeny, tu: chwyty), jak i tokeny *sterujące*. W projekcie wprowadzono trzy tokeny specjalne pochodzące z klasycznego paradygmatu *seq2seq* (Sutskever et al., 2014):

- **<SOS>** (*Start Of Sequence*) — pierwszy element wejścia dekodera w fazie inferencji. Stanowi specjalny token, którego embedding jest wyuczonym sygnałem rozpoczęcia generacji. Bez <SOS> dekodery nie miałyby zdefiniowanego warunku początkowego dla rozkładu autoregresyjnego $P(y_1 | c)$.
- **<EOS>** (*End Of Sequence*) — token oznaczający zakończenie sekwencji. W uczeniu jest dołączany na koniec każdej trasy w `tgt_out`, dzięki czemu model uczy się momentu zakończenia generacji. W inferencji wygenerowanie <EOS> powoduje zakończenie pętli dekodującej.

- **<PAD>** (*Padding*) — token zerowy używany do wyrównania długości sekwencji w batchu. Pozycje **<PAD>** są *ignorowane* przez funkcję kosztu (`ignore_index=pad_id`); w przeciwnym razie model mógłby nadmiernie dopasować się do artefaktu dopełnienia sekwencji.

W kontekście algorytmów uczenia maszynowego seq-to-seq tokeny te pełnią kilka funkcji: (i) umożliwiają tworzenie macierzy batchy o stałej długości (PAD), (ii) determinują warunki brzegowe rozkładu (SOS startuje, EOS kończy), (iii) odprzegają reprezentację końca trasy od konkretnego chwytu — bez **<EOS>** sieć musiałaby kodować koniec trasy poprzez kombinację pozycji ostatniego chwytu, co znacząco zaburzałoby semantykę słownika.

2.3 Modelowanie sekwencji warunkowych

Rozwiązywany problem należy do klasy modelowania sekwencji warunkowych (Bengio et al., 2003; Sutskever et al., 2014). Celem jest aproksymacja rozkładu:

$$P(y_1, y_2, \dots, y_T \mid c), \quad (3)$$

gdzie:

- $y_t \in \mathcal{V}$ — token w pozycji t sekwencji wyjściowej,
- T — długość sekwencji,
- c — kontekst wejściowy (warunek); tutaj para $c = (\text{layout}, \text{grade})$,
- $\mathcal{V}$ — skończony słownik tokenów.

Zgodnie z regułą łańcuchową prawdopodobieństwa łącznego:

$$P(y_{1:T} \mid c) = \prod_{t=1}^T P(y_t \mid y_{<t}, c), \quad (4)$$

gdzie $y_{<t} \equiv (y_1, \dots, y_{t-1})$ to historia tokenów do kroku $t - 1$. W praktyce model generuje kolejne tokeny trasy *autoregresyjnie*, tzn. każdy następny token zależy od historii już wygenerowanych tokenów i warunku wejściowego.

2.4 Formalizacja celu uczenia

Niech x oznacza kontekst (`layout, grade`), a $y_{1:T}$ sekwencję tokenów trasy. Uczenie przebiega przez minimalizację średniej wartości ujemnego logarytmu wiarygodności (*Negative*

Log-Likelihood, NLL):

$$\mathcal{L}_{NLL} = -\frac{1}{Z} \sum_{i=1}^N \sum_{t=1}^{T_i} \log P_{\theta}(y_t^{(i)} \mid y_{<t}^{(i)}, x^{(i)}) \cdot \mathbb{I}[y_t^{(i)} \neq \langle \text{PAD} \rangle], \quad (5)$$

gdzie:

- N — liczba próbek (par $(x^{(i)}, y^{(i)})$) w batchu/epoce,
- T_i — długość i -tej sekwencji docelowej (po dołączeniu $\langle \text{EOS} \rangle$),
- θ — parametry sieci (embeddingi, GRU, głowa wyjściowa),
- P_{θ} — rozkład warunkowy modelowany przez sieć,
- $\mathbb{I}[\cdot]$ — funkcja wskaźnikowa (maska pozycji niebędących $\langle \text{PAD} \rangle$),
- $Z = \sum_i \sum_t \mathbb{I}[y_t^{(i)} \neq \langle \text{PAD} \rangle]$ — liczba *rzeczywistych* tokenów (normalizacja).

Pominięcie pozycji $\langle \text{PAD} \rangle$ oznacza, że model jest trenowany wyłącznie na rzeczywistych pozycjach sekwencji, co eliminuje wpływ technicznego dopełnienia batchy i zapewnia, że gradient odpowiada rzeczywistej skali zadania.

2.5 Rekurencyjne sieci neuronowe (RNN, LSTM, GRU)

Klasyczne RNN. Rekurencyjne sieci neuronowe (*Recurrent Neural Networks*) (Goodfellow et al., 2016) przetwarzają sekwencje krok po kroku, utrzymując *stan ukryty* $h_t \in \mathbb{R}^h$, który podsumowuje całą historię:

$$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b_h), \quad (6)$$

gdzie:

- $x_t \in \mathbb{R}^{d_e}$ — wejście (embedding tokenu) w kroku t ,
- $W_{xh} \in \mathbb{R}^{h \times d_e}$, $W_{hh} \in \mathbb{R}^{h \times h}$ — macierze wag,
- $b_h \in \mathbb{R}^h$ — bias.

Klasyczne RNN cierpią jednak na problem *zanikającego/eksplodującego gradientu* (Pascanu et al., 2013): powtarzane mnożenie przez W_{hh} w propagacji wstecznej w czasie powoduje wykładnicze tłumienie lub eksplozję gradientu po kilkudziesięciu krokach. W praktyce uniemożliwia to uczenie zależności długodystansowych.

LSTM. Architektura *Long Short-Term Memory* (Hochreiter and Schmidhuber, 1997) rozwiązuje problem zanikania gradientu, wprowadzając trzy bramki (*input*, *forget*, *output*) i oddzielny *stan komórki* c_t :

$$\begin{aligned}
i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i), \\
f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f), \\
o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o), \\
\tilde{c}_t &= \tanh(W_c x_t + U_c h_{t-1} + b_c), \\
c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \\
h_t &= o_t \odot \tanh(c_t),
\end{aligned} \tag{7}$$

gdzie:

- $\sigma(\cdot)$ — funkcja sigmoidalna,
- $\odot$ — iloczyn Hadamarda (po współrzędnych),
- $i_t, f_t, o_t \in [0, 1]^h$ — wektory bramek wejścia, zapominania i wyjścia,
- $\tilde{c}_t \in \mathbb{R}^h$ — kandydat na nowy stan komórki,
- c_t, h_t — stan komórki i ukryty w kroku t .

LSTM charakteryzuje się wysoką skutecznością, jednak wykorzystuje cztery zestawy wag i osobny stan komórki, co zwiększa rozmiar modelu i koszt obliczeń.

GRU. *Gated Recurrent Unit* (Cho et al., 2014) jest uproszczeniem LSTM. Posiada dwie bramki: *reset* (r_t) oraz *update* (z_t), a stan komórki został zlany ze stanem ukrytym:

$$\begin{aligned}
z_t &= \sigma(W_z x_t + U_z h_{t-1} + b_z), \\
r_t &= \sigma(W_r x_t + U_r h_{t-1} + b_r), \\
\tilde{h}_t &= \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h), \\
h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t,
\end{aligned} \tag{8}$$

gdzie:

- $z_t \in [0, 1]^h$ — bramka update; steruje proporcją mieszania starego stanu i nowego kandydata,
- $r_t \in [0, 1]^h$ — bramka reset; pozwala selektywnie „zapomnieć” poprzedni stan przy obliczaniu kandydata $\tilde{h}_t$,
- W_*, U_* — macierze wagowe odpowiednich bramek.

Dlaczego GRU dla zadania Kilter? Chung et al. (2014) wykazali empirycznie, że dla sekwencji umiarkowanej długości GRU osiąga jakość porównywalną z LSTM przy mniejszej liczbie parametrów. W kontekście projektu Kilter:

1. Sekwencje tokenów trasy są *krótkie* (zwykle $T < 100$). Pełna pojemność bramki forget LSTM nie jest niezbędna.
2. Mniejsza liczba parametrów (3 macierze na bramkę zamiast 4) zmniejsza ryzyko przeczenia.
3. Mniejszy koszt obliczeniowy — około 25% szybsze trenowanie na batchach przy tej samej liczbie warstw.
4. Brak osobnego stanu komórki upraszcza implementację *warunkowania* przez inicjalizację stanu początkowego — wystarczy jeden wektor projekcji.
5. W przeciwieństwie do architektur transformatorowych GRU dobrze sprawdza się przy małej liczbie warstw i bardzo krótkim kontekście warunkującym (dwa tokeny źródłowe), gdzie self-attention nie miałby istotnej przewagi indukcyjnej.

Warunkowanie przez stan początkowy. W warstwie koncepcyjnej zastosowano architekturę *conditional decoder*: kontekst x nie jest dołączany do każdego kroku wejściowego, lecz kodowany do stanu początkowego dekodera h_0 . Dzięki temu warunek oddziałuje globalnie na całą generowaną sekwencję, a wejścia dekodera (embeddingi tokenów) niosą informację wyłącznie o historii sekwencji.

2.6 Uczenie przez teacher forcing i funkcja kosztu

W treningu zastosowano schemat *teacher forcing* (Williams and Zipser, 1989): na wejście dekodera w kroku t podawany jest *prawidłowy* token y_{t-1} (z danych uczących), a nie token przewidziany przez model. Stabilizuje to uczenie i przyspiesza zbieżność, szczególnie w początkowych epokach, gdy rozkład modelu jest jeszcze prawie jednorodny.

Jako kryterium optymalizacji wykorzystano entropię krzyżową na poziomie tokenów (NLL) z ignorowaniem pozycji dopełnienia (<PAD>). Minimalizacja NLL odpowiada maksymalizacji wiarygodności sekwencji obserwowanych w danych.

2.7 Dekodowanie probabilistyczne i wpływ parametrów generacji

W trakcie inferencji rozkład warunkowy $P_\theta(y_t \mid y_{<t}, x)$ służy do iteracyjnego losowania (lub maksymalizacji) kolejnych tokenów. Czysta strategia zachłanna (*greedy*) wybiera token o

największym prawdopodobieństwie:

$$y_t^{(\text{greedy})} = \arg \max_{y \in \mathcal{V}} P_\theta(y \mid y_{<t}, x), \quad (9)$$

ale prowadzi do *degeneracji tekstu* (Holtzman et al., 2020): powtarzających się bloków i niskiej różnorodności. W projekcie zastosowano sterowane próbkowanie z czterema parametrami; ich znaczenie i wpływ na wynik opisano poniżej.

Temperatura τ . Logity $z = (z_1, \dots, z_{|\mathcal{V}|}) \in \mathbb{R}^{|\mathcal{V}|}$ są skalowane przed normalizacją softmaxem:

$$\tilde{P}_\theta(y = v_i) = \frac{\exp(z_i/\tau)}{\sum_{j=1}^{|\mathcal{V}|} \exp(z_j/\tau)}, \quad (10)$$

gdzie:

- $\tau > 0$ — parametr temperatury,
- z_i — logit (nieznormalizowany wynik) tokenu $v_i \in \mathcal{V}$.

Dla $\tau \rightarrow 0$ rozkład zbiega do deterministycznego *argmax*; dla $\tau = 1$ używany jest natywny rozkład modelu; dla $\tau > 1$ rozkład się spłaszcza, zwiększając różnorodność kosztem precyzji.

Top- k (Fan et al., 2018). W kroku t zachowywane jest k tokenów o największych logitach, pozostałe otrzymują prawdopodobieństwo zerowe (logit ustawiany na $-\infty$):

$$\mathcal{V}_k = \text{TopK}_k(z_1, \dots, z_{|\mathcal{V}|}), \quad \tilde{P}_\theta(y) = 0 \text{ dla } y \notin \mathcal{V}_k. \quad (11)$$

Parametr k kontroluje twardy próg dopuszczalnych kandydatów; małe k ogranicza różnorodność i eliminuje mało prawdopodobne wybory, ale zbyt małe może prowadzić do nadmiernej monotoniczności.

Top- p (*nucleus sampling*) (Holtzman et al., 2020). Niech π będzie permutacją indeksów malejąco po prawdopodobieństwie, a $\hat{P}$ rozkładem po softmaxie. Definiujemy najmniejszy zbiór $\mathcal{V}_p$:

$$\mathcal{V}_p = \{v_{\pi(1)}, \dots, v_{\pi(K)}\}, \quad K = \min \left\{ k : \sum_{j=1}^k \hat{P}(v_{\pi(j)}) \geq p \right\}. \quad (12)$$

Tylko tokeny z $\mathcal{V}_p$ są kandydatami do wylosowania. W przeciwieństwie do top- k , top- p adaptacyjnie dobiera rozmiar zbioru wyboru: w obszarach wysokiej koncentracji prawdopodobieństwa zbiór jest mały, natomiast w pozycjach o większej niepewności — większy.

Repetition penalty $\rho \geq 1$. Niech $\mathcal{H}_t = \{y_1, \dots, y_{t-1}\}$ będzie zbiorem unikalnych tokenów już wygenerowanych. Dla $v \in \mathcal{H}_t$:

$$z'_v = \begin{cases} z_v / \rho & \text{jeżeli } z_v > 0, \\ z_v \cdot \rho & \text{jeżeli } z_v \leq 0, \end{cases} \quad (13)$$

co zmniejsza prawdopodobieństwo ponownego wylosowania już użytego tokenu. Im większe ρ , tym silniejsza redukcja prawdopodobieństwa powtórzeń.

Wpływ parametrów w praktyce.

- *Zwiększenie $\tau \Rightarrow$* trasy bardziej zróżnicowane, ale rośnie ryzyko nielogicznych przeskoków chwytów.
- *Zmniejszenie $\tau \Rightarrow$* trasy bardziej powtarzalne, bliższe „modalnej” trasie z danych.
- *Małe k i małe $p \Rightarrow$* wynik staje się bardziej deterministyczny, jakość pojedynczej próbki może wzrosnąć, ale generator jest bardziej podatny na lokalne pętle (przypadek Top-1).
- *Duże k / p bliskie 1 $\Rightarrow$* wynik bardziej różnorodny, ale podatny na błędy modelu w ogonie rozkładu.
- *Wysokie $\rho \Rightarrow$* ogranicza pętle, ale może wymuszać nieuzasadnione pomijanie chwytu, który naturalnie występuje wielokrotnie (np. chwyt pośrodku).

W konfiguracji projektowej przyjęto $\tau = 1,15$, $k = 40$, $p = 0,95$, $\rho = 1,1$, co empirycznie zapewnia akceptowalny kompromis pomiędzy różnorodnością a poprawnością.

3 Realizacja sieci

3.1 Reprezentacja danych i przygotowanie batchy

Dane wejściowe są zapisane jako listy identyfikatorów całkowitoliczbowych. Każda próbka zawiera:

- **source:** dwa tokeny (`layout_id`, `grade_id`),
- **target:** sekwencję tokenów trasy.

W preprocessingu tworzona jest osobna indeksacja dla layoutów i poziomów trudności (`layout_to_idx`, `grade_to_idx`), co zapewnia zwarte zakresy indeksów dla embeddingów warunkowych.

Funkcja `collate_batch` buduje dwa warianty sekwencji docelowej:

- $\text{tgt_in} = [\text{<SOS>}, t_1, \dots, t_n],$
- $\text{tgt_out} = [t_1, \dots, t_n, \text{<EOS>}],$

następnie dopełnia obie macierze do maksymalnej długości w batchu tokenem <PAD>.

```
for t in targets:
    tgt_in = torch.cat([torch.tensor([sos_id]), t], dim=0)
    tgt_out = torch.cat([t, torch.tensor([eos_id])], dim=0)
    tgt_in_list.append(tgt_in)
    tgt_out_list.append(tgt_out)
```

Listing 2: Budowa par $\text{tgt_in}/\text{tgt_out}$ (collate_batch).

Powyższa konstrukcja realizuje standardowe przesunięcie sekwencji o jeden krok (ang. *next-token prediction*), co jest podstawą uczenia autoregresyjnego.

3.2 Architektura modelu

Model `KilterRouteGenerator` składa się z trzech embeddingów:

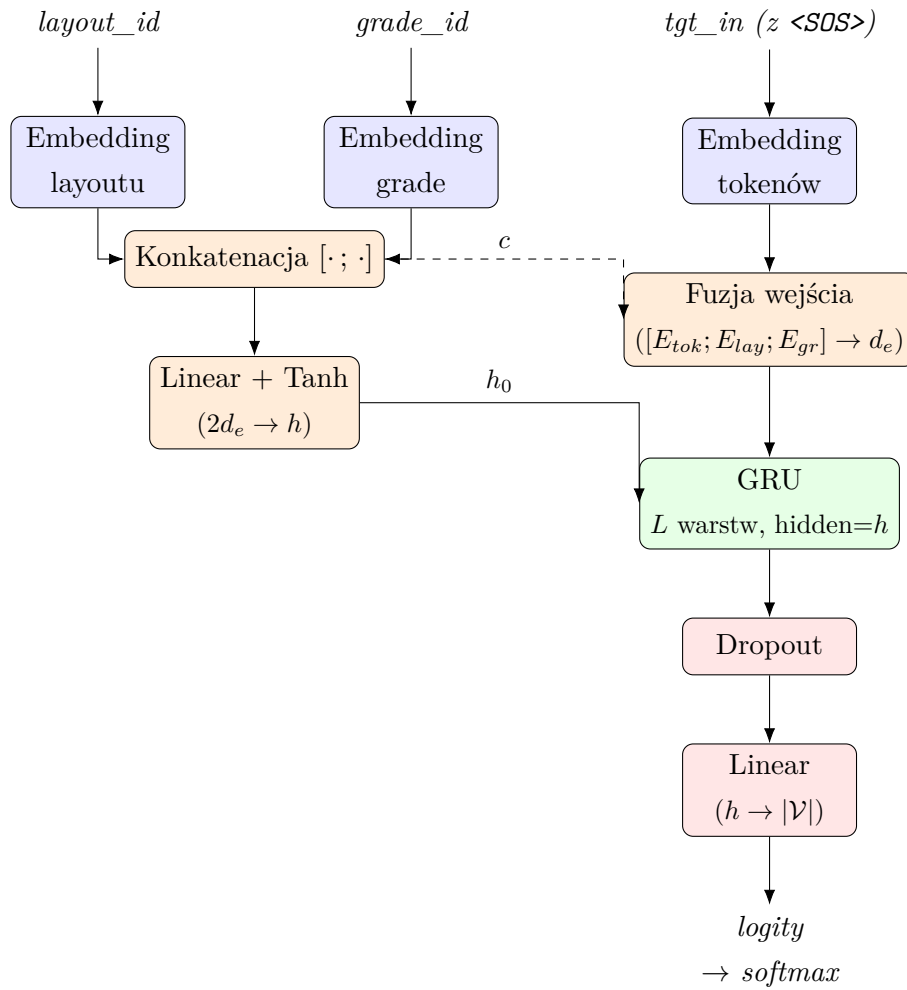
- embedding tokenów sekwencji $E_{\text{tok}} \in \mathbb{R}^{|\mathcal{V}| \times d_e},$
- embedding layoutu $E_{\text{lay}} \in \mathbb{R}^{|\mathcal{L}| \times d_e},$
- embedding grade $E_{\text{gr}} \in \mathbb{R}^{|\mathcal{G}| \times d_e},$

gdzie:

- d_e — wymiar embeddingu (w eksperymencie $d_e = 128$),
- $|\mathcal{L}|, |\mathcal{G}|$ — rozmiary słowników layoutów i ocen trudności,
- h — wymiar stanu ukrytego GRU (w eksperymencie $h = 256$).

Embedding layoutu i grade są konkatenowane i projekcjonowane liniowo do przestrzeni stanu ukrytego przez warstwę **Linear+Tanh**. Otrzymany wektor pełni rolę stanu początkowego dekodera GRU i jest kopiowany na wszystkie warstwy rekurencyjne. W poprawionej wersji modelu ten sam kontekst jest również opcjonalnie dołączany do każdego kroku wejściowego dekodera (`condition_every_step`). Dzięki temu informacja o układzie panelu i trudności nie musi być przechowywana wyłącznie w stanie początkowym h_0 , co ogranicza jej stopniowe zanikanie przy dłuższych generacjach.

Schematycznie działanie modelu prezentuje rysunek 5.



Rysunek 5: Architektura warunkowego dekodera GRU KilterRouteGenerator. Para (layout_id, grade_id) koduje stan początkowy h_0 oraz, w ulepszonym wariancie, jest dołączana do wejścia każdego kroku dekodera.

Sekwencja `tgt_in` przechodzi przez dekodera GRU, po czym stosowany jest dropout i warstwa liniowa wyjściowa mapująca stan ukryty na logity po całym słowniku tokenów.

```

self.token_embedding = nn.Embedding(vocab_size, embed_dim)
self.layout_embedding = nn.Embedding(layout_vocab_size, embed_dim)
self.grade_embedding = nn.Embedding(grade_vocab_size, embed_dim)
self.context_proj = nn.Sequential(
    nn.Linear(embed_dim * 2, hidden_dim),
    nn.Tanh(),
)
self.input_fusion = nn.Sequential(
    nn.Linear(embed_dim * 3, embed_dim),
    nn.Tanh(),
) # gdy condition_every_step=True
self.decoder = nn.GRU(input_size=embed_dim, hidden_size=hidden_dim,
                      num_layers=num_layers, batch_first=True)
  
```

```
self.head = nn.Linear(hidden_dim, vocab_size)
```

Listing 3: Definicja modelu — moduł architektury.

```
layout_vec = self.layout_embedding(source[:, 0])
grade_vec = self.grade_embedding(source[:, 1])
context = torch.cat([layout_vec, grade_vec], dim=-1)
hidden0 = self.context_proj(context).unsqueeze(0)
return hidden0.repeat(self.decoder.num_layers, 1, 1)
```

Listing 4: Inicjalizacja stanu ukrytego z warunku — moduł architektury.

```
token_vec = self.token_embedding(tgt_input)
context = self.context_embedding(source).unsqueeze(1)
context = context.expand(-1, tgt_input.shape[1], -1)
decoder_input = self.input_fusion(torch.cat([token_vec, context], dim=-1))
```

Listing 5: Fuzja kontekstu z tokenem wejściowym w każdym kroku dekodera.

Interpretacyjnie: model uczy się funkcji $f : \mathbb{R}^{2d_e} \rightarrow \mathbb{R}^h$, która mapuje semantykę warunku (layout + grade) na punkt startowy dynamiki GRU. Ulepszona ścieżka `input_fusion` dodatkowo uczy funkcji $g : \mathbb{R}^{3d_e} \rightarrow \mathbb{R}^{d_e}$, dzięki której każdy krok dekodera widzi zarówno historię tokenów, jak i jawny kontekst zadania. To zmniejsza ryzyko, że przy dłuższej sekwencji model zacznie generować trasę zgodną lokalnie z historią, ale słabiej dopasowaną do żądanej trudności lub layoutu.

3.3 Procedura uczenia

Uczenie realizowane jest optymalizatorem AdamW (sekcja 3.4) z clippingiem gradientu ($\|g\|_2 \leq 1,0$) (Pascanu et al., 2013). Dla każdej epoki obliczane są:

- `train_nll` — strata na zbiorze treningowym,
- `val_nll` — strata walidacyjna,
- `val_ppl` — perplexity, $\exp(\text{val_nll})$.

Najlepszy checkpoint jest zapisywany według kryterium minimalnego `val_nll`, co ogranicza ryzyko nadmiernego dopasowania i pozwala zachować najbardziej użyteczną wersję modelu do generacji.

```
logits = model(src, tgt_in)
loss = F.cross_entropy(
    logits.reshape(-1, logits.shape[-1]),
    tgt_out.reshape(-1),
```

```

        ignore_index=pad_id,
        reduction="sum",
    )
token_count = (tgt_out != pad_id).sum().item()
(loss / max(1, token_count)).backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
optimizer.step()

```

Listing 6: Krok optymalizacji — moduł treningowy.

Istotnym szczegółem implementacyjnym jest normalizacja straty przez liczbę rzeczywistych tokenów (`token_count`), a nie przez długość dopełnionego batcha. Dzięki temu estymator gradientu pozostaje porównywalny między batchami o różnych długościach sekwencji.

W badanym przebiegu (`report_full`) wykorzystano m.in. parametry: `epochs=50`, `batch_size=32`, `lr=0.001`, `embed_dim=128`, `hidden_dim=256`, `layers=3`, `dropout=0.15`, urządzenie `cuda`. Nowe treningi domyślnie włączają `condition_every_step=True`; starsze checkpointy pozostają kompatybilne, ponieważ parametr architektury jest zapisywany w konfiguracji modelu.

3.4 Optymalizator AdamW

Geneza. AdamW (Loshchilov and Hutter, 2019) jest modyfikacją klasycznego optymalizatora Adam (Kingma and Ba, 2015). Adam łączy momentum (pierwszy moment gradientu) z adaptacyjnym skalowaniem kroku per parametr (drugi moment). Loshchilov and Hutter (2019) zauważyli, że tradycyjna implementacja *weight decay* w Adamie (dodanie członu $\lambda\theta$ do gradientu) splata regularyzację z momentum, co odbiega od formalnej definicji L_2 -regularyzacji. AdamW *rozprzega* weight decay: jest on stosowany po aktualizacji adaptacyjnej, bezpośrednio na wagach.

Algorytm. Niech $g_t \in \mathbb{R}^{|\theta|}$ będzie gradientem straty w kroku t .

Algorithm 1 AdamW (Decoupled Weight Decay) — (Loshchilov and Hutter, 2019)

Require: parametry θ_0 , krok uczenia η , współczynniki momentum $\beta_1, \beta_2 \in [0, 1)$, weight decay $\lambda \geq 0$, stała numeryczna $\epsilon > 0$

```
1:  $m_0 \leftarrow 0, v_0 \leftarrow 0, t \leftarrow 0$ 
2: while nie osiągnięto kryterium stopu do
3:    $t \leftarrow t + 1$ 
4:    $g_t \leftarrow \nabla_{\theta} \mathcal{L}(\theta_{t-1})$  ▷ gradient mini-batcha
5:    $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$  ▷ pierwszy moment
6:    $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t \odot g_t$  ▷ drugi moment
7:    $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$  ▷ korekta obciążenia
8:    $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ 
9:    $\theta_t \leftarrow \theta_{t-1} - \eta \left( \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right)$  ▷ aktualizacja + odsprężony decay
10: end while
11: return  $\theta_t$ 
```

Symbole:

- θ_t — parametry sieci w kroku t ,
- g_t — gradient w kroku t ,
- m_t — estymator wykładniczo ważonej średniej gradientów (momentum),
- v_t — estymator wykładniczo ważonej średniej kwadratów gradientu (skala),
- $\hat{m}_t, \hat{v}_t$ — estymatory skorygowane o obciążenie (*bias correction*),
- η — krok uczenia (1r); $\beta_1 = 0,9$, $\beta_2 = 0,999$ (wartości typowe),
- λ — współczynnik weight decay; $\epsilon \approx 10^{-8}$ — stabilność numeryczna.

Zalety w kontekście projektu.

- **Adaptacyjny krok per parametr.** Embeddingi rzadko aktualizowanych tokenów (np. rzadkie layouty) otrzymują większą efektywną stopę uczenia niż gęsto używane macierze GRU.
- **Odsprężony weight decay.** Zapewnia spójną regularyzację *niezależną* od skali drugiego momentu — empirycznie poprawia uogólnienie (Loshchilov and Hutter, 2019).
- **Stabilność w mieszanych skalach.** W naszej architekturze współistnieją małe wagi głowy wyjściowej i większe wagi GRU; adaptacyjna skala zapobiega temu, by małe wagi były „zdominowane” przez duże.

- **Odporność względem doboru learning rate.** W projekcie użyto $\eta = 10^{-3}$, co jest typowym i stabilnym wyborem dla AdamW na zadaniach językowych.
- **Małe wymagania pamięciowe** ($2|\theta|$ dodatkowych zmiennych — m, v), znacznie mniejsze niż dla metod drugiego rzędu.

3.5 Dobór zbioru walidacyjnego a walidacja krzyżowa

W badaniach przyjęto *trzymany* zbiór walidacyjny (*held-out validation set*) o wielkości 1% wszystkich próbek, czyli $|\mathcal{D}_{\text{val}}| = 2\,227$ tras, podczas gdy $|\mathcal{D}_{\text{train}}| = 220\,397$. Wybór ten został podjęty świadomie i zasługuje na komentarz w kontekście klasycznej walidacji krzyżowej.

Walidacja krzyżowa — przypomnienie. W k -krotnej walidacji krzyżowej (*k-fold cross-validation*) (Stone, 1974; Kohavi, 1995) zbiór danych $\mathcal{D}$ dzielony jest na k rozłącznych podzbiorów $\mathcal{D}^{(1)}, \dots, \mathcal{D}^{(k)}$. Model trenowany jest k razy — za każdym razem na $\mathcal{D} \setminus \mathcal{D}^{(j)}$, ewaluowany na $\mathcal{D}^{(j)}$. Estymator błędu:

$$\widehat{\text{CV}}_k = \frac{1}{k} \sum_{j=1}^k \mathcal{L}_{\text{val}}(\theta^{(j)}; \mathcal{D}^{(j)}), \quad (14)$$

gdzie:

- k — liczba foldów (typowo 5 lub 10),
- $\theta^{(j)}$ — parametry modelu wytrenowanego z pominięciem j -tego foldu,
- $\mathcal{L}_{\text{val}}$ — funkcja kosztu na zbiorze walidacyjnym.

$\widehat{\text{CV}}_k$ ma mniejszą wariancję niż pojedynczy split, ale wymaga k -krotnego trenowania modelu.

Dlaczego nie zastosowano k -fold CV? Decyzja o pojedynczym, ustalonym podziale danych została podjęta z następujących powodów:

1. **Skala danych.** Korpus liczy $N = 222\,624$ tras. Przy tak dużym N pojedyncza walidacja jest już dobrym estymatorem błędu generalizacji — wariancja estymatora $\mathcal{L}_{\text{val}}$ malejąca jak $\mathcal{O}(1/|\mathcal{D}_{\text{val}}|)$ jest dostatecznie mała przy $|\mathcal{D}_{\text{val}}| > 2000$.
2. **Koszt obliczeniowy.** Jedna pełna epoka trwa kilka minut na GPU; pełne 5-fold CV przy 50 epokach wymagałoby $5 \times 50 = 250$ przebiegów, co przy każdej zmianie konfiguracji byłoby nieproporcjonalne względem oczekiwanego zysku informacyjnego.

3. **Cel: model, nie estymator.** Celem nie jest *estymacja błędu generalizacji* z najwyższą możliwą precyzją, lecz *wytrenowanie najlepszego dostępnego modelu* oraz użycie zbioru walidacyjnego jako kryterium wczesnego zatrzymania (*early stopping*) i selekcji najlepszego checkpointu.
4. **Brak rozbudowanego przeszukiwania hiperparametrów.** Hiperparametry zostały dobrane na podstawie literatury (rozmiary embeddingu, dropout, learning rate); *k*-fold CV jest szczególnie użyteczne przy szerokim *grid search*, którego w niniejszej pracy nie prowadzono.
5. **Wewnętrzna różnorodność.** Próbkę są losowo tasowane (`random.shuffle(samples)` z ustalonym `seed=42`) przed splitem, co czyni zbiór walidacyjny reprezentatywnym względem rozkładu łącznego (warstwy trudności, layouty).

Kiedy warto przejść na *k*-fold? W rozszerzonej fazie projektu (poza zakresem niniejszego raportu) wskazane byłoby:

- **Stratified *k*-fold** po atrybucie `grade_id` — by zagwarantować, że rzadkie trudności są reprezentowane w każdym foldzie,
- **Grouped CV** po `layout_id` — gdy celem jest ocena uogólnienia modelu na *niewidziane* układy panelu,
- **Repeated holdout** — kompromis o niższym koszcie obliczeniowym niż pełna *k*-fold CV.

W obecnej fazie pojedynczy holdout 99%/1% pozostaje uzasadnionym kompromisem między precyzją szacunku, kosztem obliczeń i celami praktycznymi projektu.

3.6 Generacja i postprocessing

Po wczytaniu checkpointu model przyjmuje wejście [`layout_id`, `grade_id`], mapuje je do indeksów treningowych i generuje sekwencję od tokenu `<SOS>` aż do `<EOS>` lub do osiągnięcia limitu długości. Wygenerowane tokeny specjalne są odfiltrowywane, a tokeny pozycji chwyków (np. `p123`) są konwertowane do listy numerów otworów panelu.

```
generated = model.generate(  
    source=source,  
    sos_id=ckpt["sos_id"],  
    eos_id=ckpt["eos_id"],  
    max_len=args.max_len,  
    do_sample=not args.greedy,  
    temperature=args.temperature,  
    top_k=args.top_k,
```

```
top_p=args.top_p,  
repetition_penalty=args.repetition_penalty,  
)
```

Listing 7: Konfiguracja dekodowania — moduł inferencji.

Z punktu widzenia użytkowego mechanizm ten umożliwia sterowanie kompromisem pomiędzy deterministycznością a różnorodnością generacji, bez konieczności modyfikowania parametrów samej sieci (patrz sekcja 2.7). Tak przygotowany wynik może być bezpośrednio użyty przez kolejne moduły systemu (np. wizualizację/integrację), niezależnie od samego procesu uczenia.

4 Wyniki eksperymentów

4.1 Stanowisko eksperymentalne

W celu pomiaru wpływu pojedynczych hiperparametrów uczenia oraz parametrów dekodowania zrealizowano dwa zestawy scenariuszy zamknięte w skrypcie scenariuszy eksperymentalnych (rozdz. 1.5):

- **Grupa A (uczenie)** — 11 niezależnych przebiegów treningowych, w których zmieniano po jednym hiperparametrze względem konfiguracji bazowej (`A0_baseline`). Każdy przebieg zapisuje metryki epokowe oraz podsumowanie konfiguracji do osobnego katalogu wewnątrz katalogu logów eksperymentów.
- **Grupa B (generacja)** — 14 wywołań inferencji z tym samym checkpointem bazowym (`exp_A0_baseline.pt`) i tym samym promptem (`layout = 1272, grade = 1240`), czyli „12×12 without keyboard Square” / „6a+/V3”. Wyniki trafiają do katalogu próbek (po jednym pliku JSON na scenariusz).

Z uwagi na potrzebę szybkiego porównania trendów w grupie A użyto ograniczonego podzbioru (`MAX_SAMPLES=20 000`, `EPOCHS=5`, `VAL_RATIO=0,05`, urządzenie CPU). Konfiguracja `report_full` z sekcji 1.2 jest niezależna i służyła do walidacji jakościowej najlepszego modelu.

Wszystkie wykresy w niniejszej sekcji powstają bezpośrednio z surowych logów eksperymentów — za ich produkcję odpowiada moduł raportowy (rozdz. 1.5): generator wykresów renderuje rysunki do formatu PDF (wektorowo), a skrypt uruchomieniowy integruje cały proces w jedno polecenie.

4.2 Wpływ parametrów uczenia

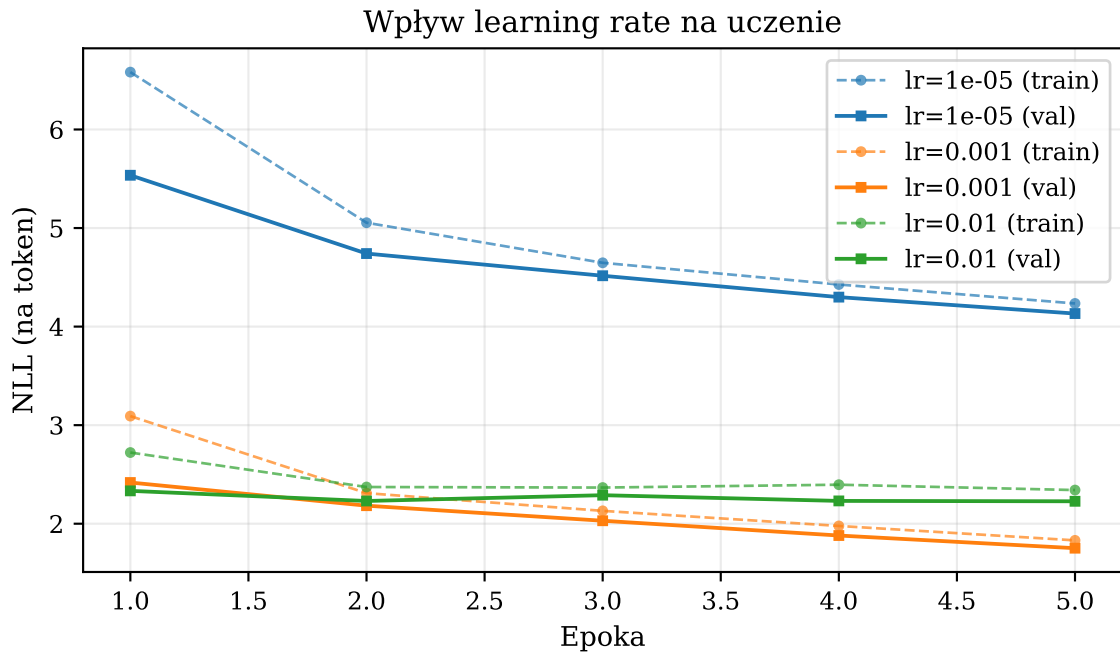
Tabela 1 zestawia konfiguracje grupy A wraz z najlepszą wartością NLL na zbiorze walidacyjnym (`best_val_nll`), perplexity oraz średnim czasem epoki. Pełne krzywe uczenia

w funkcji epoki przedstawiono na rysunkach 6–10, a zbiorcze rankingi — na rysunkach 11 i 12.

Tabela 1: Konfiguracje przebiegów treningowych (grupa A) oraz wyniki na walidacji. Wszystkie konfiguracje korzystają z optymalizatora AdamW z gradient clippingiem $\|g\|_2 \leq 1,0$.

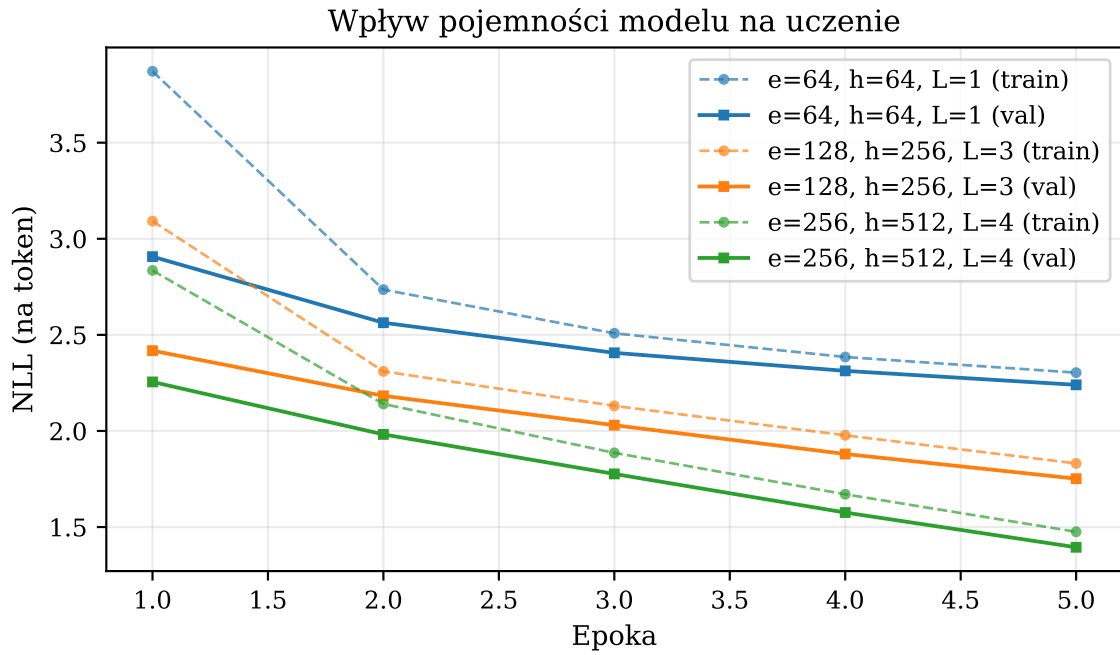
Eksperyment	lr	embed	hidden	L	dropout	bs	best val_nll	val_ppl	$\overline{t_{ep}}$ [s]
A0_baseline	10^{-3}	128	256	3	0.15	64	1.752	5.76	41.9
A1_lr_low	10^{-5}	128	256	3	0.15	64	4.132	62.33	41.2
A1_lr_high	10^{-2}	128	256	3	0.15	64	2.228	9.28	44.8
A2_small_model	10^{-3}	64	64	1	0.15	64	2.240	9.39	7.8
A2_large_model	10^{-3}	256	512	4	0.15	64	1.395	4.03	141.2
A3_dropout_off	10^{-3}	128	256	3	0.00	64	1.658	5.25	37.1
A3_dropout_high	10^{-3}	128	256	3	0.50	64	1.928	6.87	46.2
A4_batch_small	10^{-3}	128	256	3	0.15	16	1.337	3.81	47.6
A4_batch_large	10^{-3}	128	256	3	0.15	256	2.192	8.95	55.7
A5_layers_1	10^{-3}	128	256	1	0.15	64	1.645	5.18	19.0
A5_layers_5	10^{-3}	128	256	5	0.15	64	1.902	6.70	66.6

Learning rate. Rysunek 6 wskazuje, że $lr = 10^{-5}$ skutkuje niedouczeniem — po 5 epokach $val_nll \approx 4,13$, czyli model pozostaje zbliżony do początkowego rozkładu jednorodnego. Z drugiej strony $lr = 10^{-2}$ wprowadza wyraźną niestabilność: w epokach 2–4 strata walidacyjna oscyluje (2.23 / 2.29 / 2.23), co obrazuje typowy efekt zbyt dużego kroku w AdamW. Wartość bazowa 10^{-3} prowadzi do najbardziej stabilnego przebiegu uczenia w tej grupie eksperymentów.



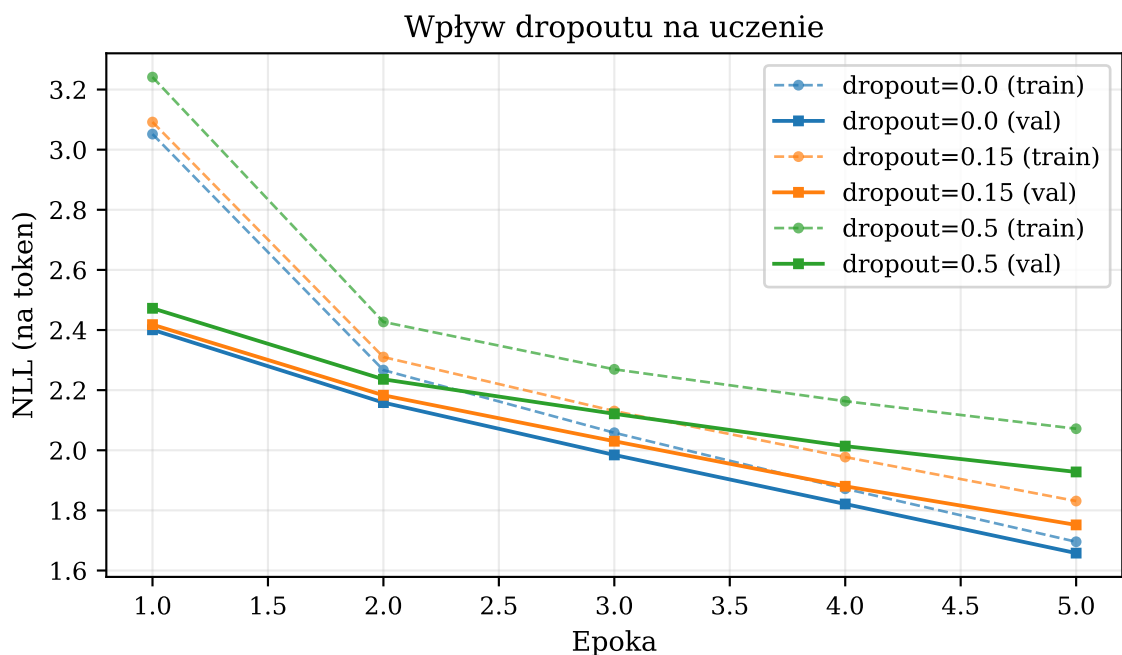
Rysunek 6: Krzywe `train_nll` (linie przerywane) i `val_nll` (linie ciągłe) dla różnych wartości `lr`.

Pojemność modelu. Wzrost wymiarów (`embed_dim`, `hidden_dim`) i głębokości (`layers`) przekłada się na lepsze dopasowanie. Konfiguracja `A2_large` osiąga `val_nll`= 1,395 (najlepszy wynik wśród modeli porównywalnej długości treningu), ale przy ok. $3,4\times$ dłuższym czasie epoki. Model o mniejszej pojemności (`A2_small`) jest $5\times$ szybszy, lecz krzywa uczenia wskazuje na ograniczenia jego pojemności reprezentacyjnej (rys. 7).



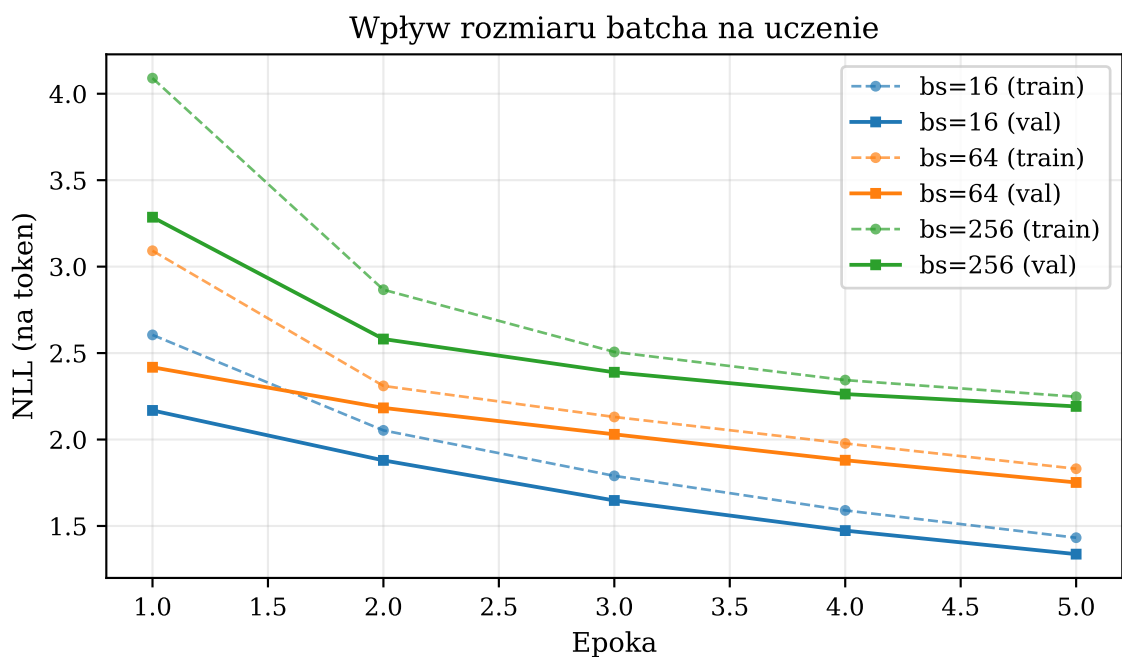
Rysunek 7: Wpływ pojemności modelu (rozmiary embeddingu, stanu ukrytego i liczby warstw).

Dropout. Na podzbiorze 20 000 próbek włączenie `dropout=0,5` powoduje wyraźne pogorszenie (`val_nll` 1,93), a brak dropoutu skutkuje minimalnie lepszym wynikiem niż konfiguracja bazowa (1.66 vs 1.75). Sugeruje to, że przy ograniczonym budżecie danych i krótkim treningu dropout 0.5 nadmiernie tłumi sygnał. Dla pełnego korpusu 222 624 próbek umiarkowany dropout (0.15) jest natomiast uzasadniony — chroni przed przeuczeniem przy dużej liczbie epok.



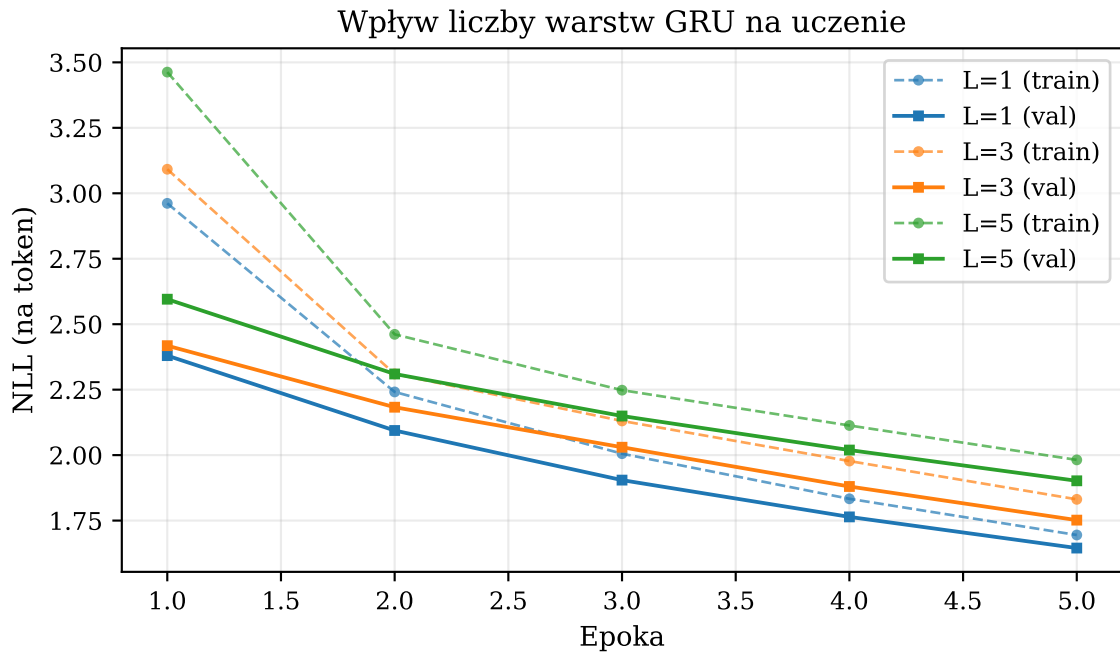
Rysunek 8: Wpływ dropoutu na proces uczenia.

Rozmiar batcha. Najbardziej korzystny wynik uzyskuje `A4_batch_small` (`bs= 16`): `val_nll= 1,337`, co stanowi najlepszy wynik w grupie A poza modelem dużym. Wynika to z większej liczby kroków optymalizatora w epoce (1188 zamiast 297 przy `bs= 64` i 74 przy `bs= 256`) oraz silniej stochastycznego gradientu, który pełni rolę regularyzatora (Loshchilov and Hutter, 2019). Duży batch (`bs= 256`) wyraźnie pogarsza wynik (`val_nll 2,19`).



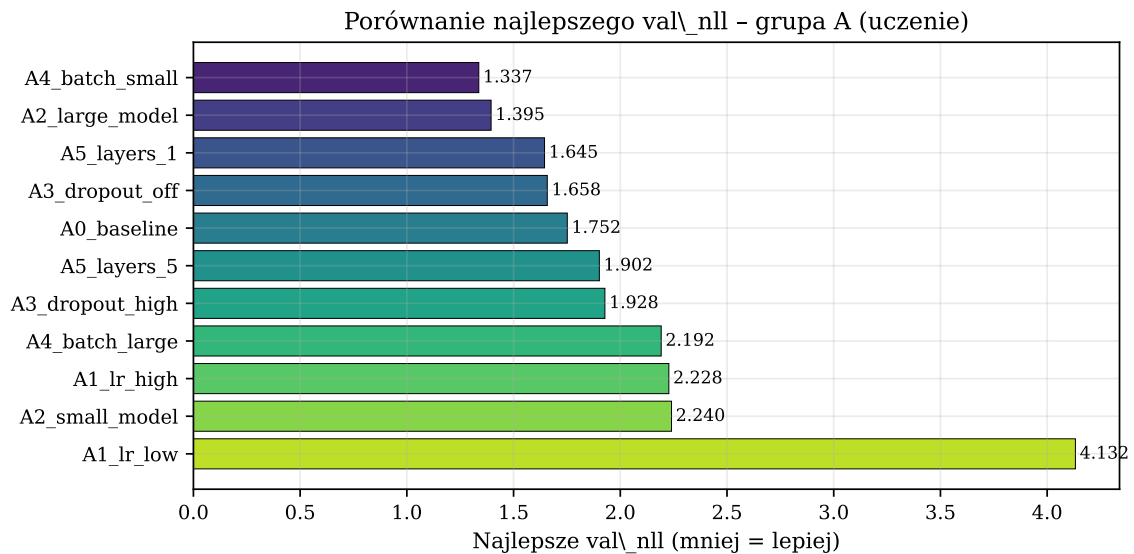
Rysunek 9: Wpływ rozmiaru batcha na zbieżność i jakość walidacji.

Liczba warstw GRU. Wbrew oczekiwaniom, `A5_layers_1` (jedna warstwa) osiąga `val_nll= 1,65`, czyli wynik lepszy niż baseline z trzema warstwami (1.75) oraz wariant pięciowarstwowy (1.90). Dla zadania o krótkich sekwencjach ($T < 100$) i prostym kontekście warunkującym (2 tokeny) głęboki dekoderek rekurencyjny nie poprawia wyników i wymaga więcej epok do efektywnego przekazania gradientu wstecznego. Zależność tę przedstawiają krzywe na rys. 10.

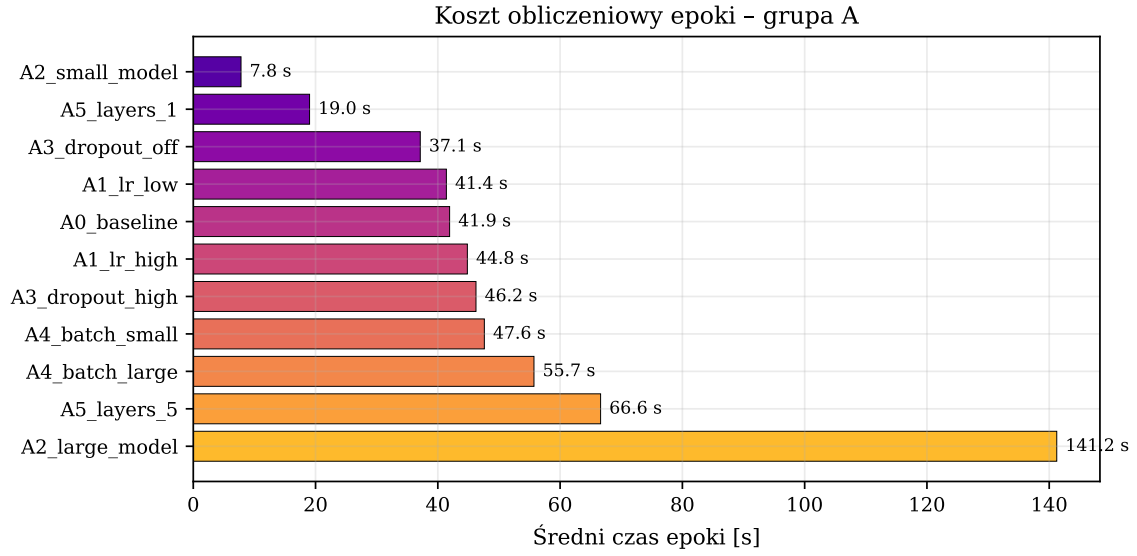


Rysunek 10: Wpływ liczby warstw GRU na proces uczenia.

Zbiórce porównanie. Rysunki 11 i 12 zestawiają najlepsze `val_nll` oraz średni czas epoki dla wszystkich konfiguracji grupy A.



Rysunek 11: Ranking najlepszego `val_nll` (mniej = lepiej) dla 11 konfiguracji uczenia.



Rysunek 12: Średni czas epoki dla 11 konfiguracji uczenia (CPU).

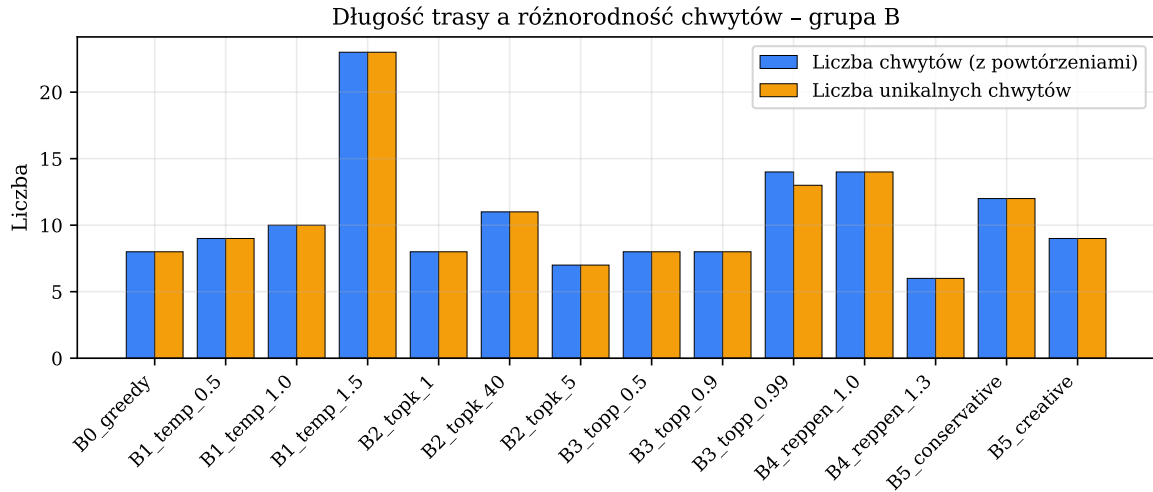
4.3 Wpływ parametrów generacji

W grupie B utrzymujemy stały checkpoint bazowy i zmieniamy parametry dekodowania. Tabela 2 zestawia 14 scenariuszy oraz dwie podstawowe metryki na poziomie tokenów:

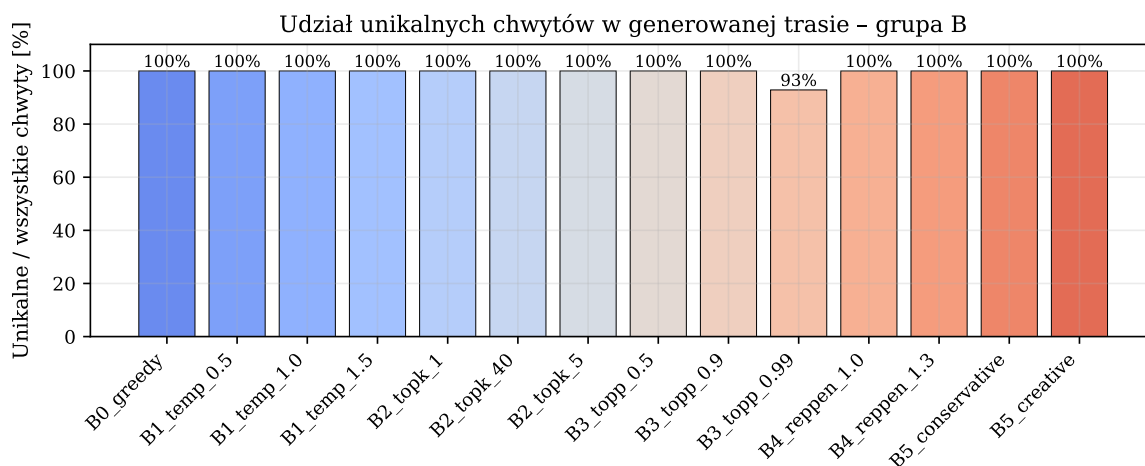
- n_h — liczba chwyków (tokenów p<id>) wygenerowanych przed <EOS> ,
- n_h^{uniq}/n_h — udział unikalnych chwyków (rośnie, gdy model nie powtarza chwyków).

Tabela 2: Konfiguracje dekodowania (grupa B) i statystyki wygenerowanej trasy dla promptu (`layout` = 1272, `grade` = 1240). Pole „—” oznacza, że dany parametr nie ma znaczenia w danym wariancie (greedy ignoruje wszystkie parametry losowe).

Eksperyment	τ	top- k	top- p	ρ	n_h	n_h^{uniq}/n_h
B0_greedy	—	—	—	—	8	100%
B1_temp_0.5	0.5	0	1.00	1.0	9	100%
B1_temp_1.0	1.0	0	1.00	1.0	10	100%
B1_temp_1.5	1.5	0	1.00	1.0	23	100%
B2_topk_1	1.0	1	1.00	1.0	8	100%
B2_topk_5	1.0	5	1.00	1.0	7	100%
B2_topk_40	1.0	40	1.00	1.0	11	100%
B3_topp_0.5	1.0	0	0.50	1.0	8	100%
B3_topp_0.9	1.0	0	0.90	1.0	8	100%
B3_topp_0.99	1.0	0	0.99	1.0	14	92.9%
B4_reppen_1.0	1.0	40	0.95	1.0	14	100%
B4_reppen_1.3	1.0	40	0.95	1.3	6	100%
B5_conservative	0.7	10	0.90	1.1	12	100%
B5_creative	1.4	60	0.98	1.05	9	100%

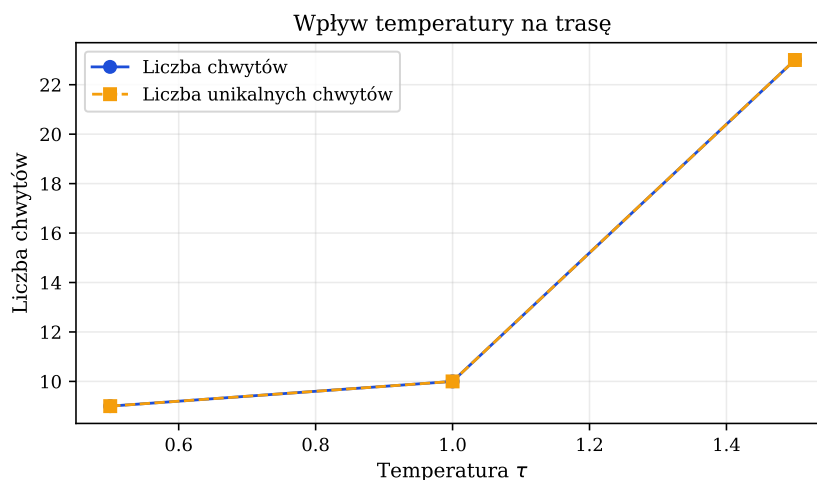


Rysunek 13: Liczba chwytów i liczba unikalnych chwytów w trasie wygenerowanej w każdym scenariuszu B.



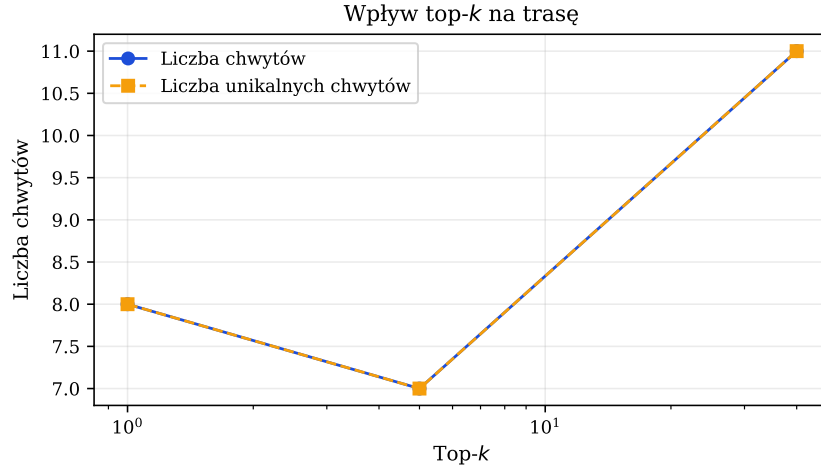
Rysunek 14: Udział unikalnych chwytów w trasie. Wartość $< 100\%$ świadczy o powtórzeniach.

Temperatura τ (rys. 15). Wzrost τ z 0,5 przez 1,0 do 1,5 wydłuża trasę z 9 do 10, a następnie do 23 chwytów. Trasa z $\tau = 1,5$ traci jednak *strukturę gramatyczną* (sekcja 5): pomiędzy tokenami p1107, p1078, p1113, p1119, p1091 brakuje tokenów ról, a w generacji pojawia się rzadki rolowy token r21. To klasyczny przykład degeneracji wysokotemperaturowej (Holtzman et al., 2020).



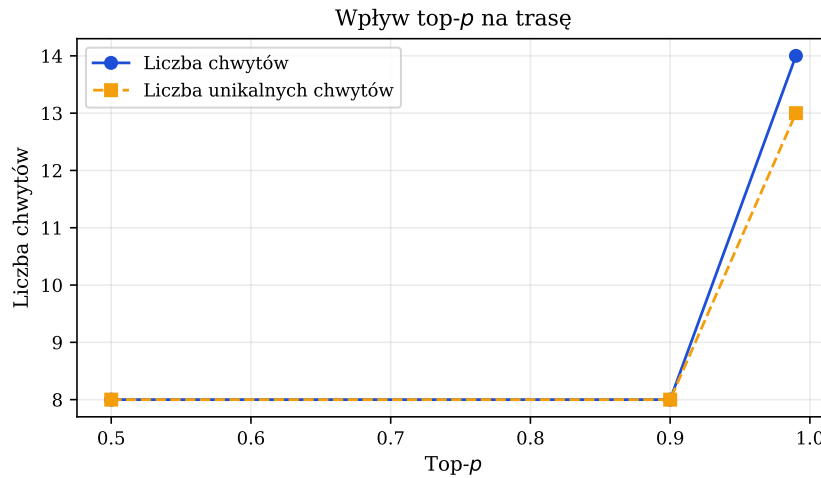
Rysunek 15: Wpływ temperatury τ na długość i różnorodność trasy.

Top- k (rys. 16). Dla $k = 1$ próbkowanie redukuje się efektywnie do greedy — ten sam wynik co B0 (8 chwytów). Bardzo małe $k = 5$ skraca trasę do 7 chwytów, gdyż model bardzo szybko trafia na token $\langle \text{EOS} \rangle$. Większe $k = 40$ pozwala na 11 chwytów — bliższe długości typowych tras V3 z korpusu.



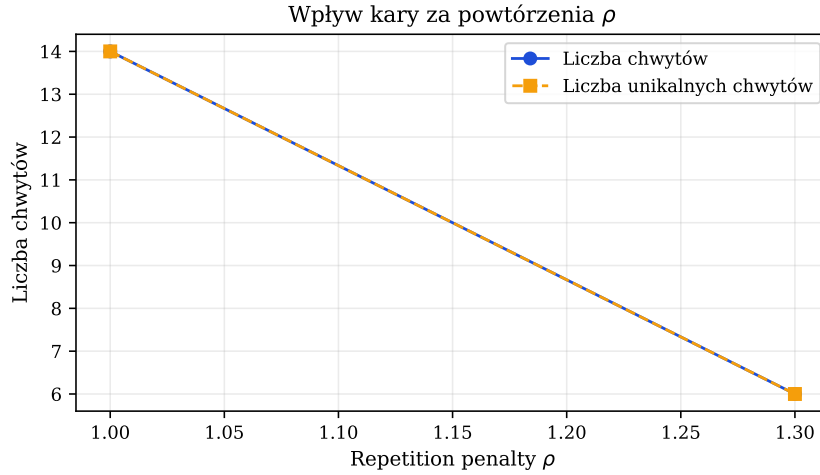
Rysunek 16: Wpływ rozmiaru zbioru kandydatów top- k .

Top- p (rys. 17). Top- $p = 0,5$ i $p = 0,9$ generują w tym przypadku identycznie krótkie trasy ($n_h = 8$). Dopiero $p = 0,99$ dopuszcza wystarczająco szeroki ogon rozkładu, aby trasa wydłużyła się do 14 chwyków, lecz jednocześnie pojawia się powtórzenie chwytu p1285 (ratio $13/14 = 92,9\%$). Ilustruje to kompromis *coverage-quality*: zwiększanie p zwiększa różnorodność, ale dopuszcza tokeny z mniej wiarygodnej części rozkładu.



Rysunek 17: Wpływ progu *nucleus sampling* top- p .

Repetition penalty ρ (rys. 18). Wyłączenie kary ($\rho = 1,0$) prowadzi do trasy 14-chwytowej bez powtórzeń, co oznacza, że w tym scenariuszu dodatkowa penalizacja nie była konieczna. Wysoka wartość kary $\rho = 1,3$ skraca trasę o ponad połowę (do 6 chwyków), gdyż token `<EOS>` uzyskuje relatywnie wyższe prawdopodobieństwo względem chwyków, które nie zostały jeszcze wykorzystane. Wskazuje to, że ρ oddziałuje pośrednio także na długość generacji, nie tylko na samo unikanie powtórzeń.



Rysunek 18: Wpływ kary za powtórzenia ρ .

Profil zachowawczy vs eksploracyjny. Konfiguracje B5 ilustrują wpływ jednoczesnej zmiany kilku parametrów dekodowania. B5_conservative ($\tau = 0,7$, $k = 10$, $p = 0,9$, $\rho = 1,1$) generuje 12 chwytów — długość zbliżoną do typowych tras V3 z korpusu. B5_creative ($\tau = 1,4$, $k = 60$, $p = 0,98$, $\rho = 1,05$) generuje 9 mniej typowych chwytów; pojawia się mniej prawdopodobny token początkowy p1157 w roli r15 (start) zamiast częściej obserwowanych p1107 / p1181.

5 Analiza jakości i ograniczenia

5.1 Mocne strony modelu

Z analizy wygenerowanych tras wynika, że nawet stosunkowo prosty warunkowy dekodnik GRU odwzorował trzy istotne cechy rozkładu:

1. **Gramatyka przeplotu p<id>/r<role>.** W każdym scenariuszu B z umiarkowanymi parametrami ($\tau \leq 1,0$, $p \leq 0,9$) model generuje ścisłą sekwencję „chwyt — rola — chwyt — rola...”. Taka gramatyka nie była wymuszona algorytmicznie, lecz wyłoniła się z danych.
2. **Monotoniczne wznoszenie się.** Identyfikatory chwytów w wygenerowanych trasach niemal monotonicznie rosną wzdłuż sekwencji (np. B0_greedy: 1154, 1181, 1219, 1235, 1285, 1304, 1353, 1390). Numeracja otworów jest skorelowana z wysokością na panelu, więc model odtworzył fizyczny porządek ścieżki „od dołu do góry”.
3. **Poprawne użycie ról.** Tokeny r15 (start), r12/r13 (chwyt pośrodku / na nogi), r14 (top) pojawiają się we właściwych miejscach trasy — r14 zwykle blisko końca sekwencji.

5.2 Obserwowane błędy

Degeneracja wysokotemperaturowa. W B1_temp_1.5 model narusza gramatykę sekwencji (brak ról między czterema kolejnymi tokenami pozycji). Generuje także rzadko spotykany token `r21`, prawdopodobnie odpowiadający kategorii roli słabo reprezentowanej w korpusie.

Powtórzenia w ogonie rozkładu. W B3_topp_0.99 pojawia się duplikat chwytu `p1285`. Bez aktywnej kary za powtórzenia ($\rho = 1,0$) model sam korzysta z tej samej pozycji w sekwencji, mimo że w typowej trasie z korpusu chwyt nie wystąpi dwa razy.

Niestabilność uczenia przy zbyt dużym lr. W A1_lr_high ($\text{lr} = 10^{-2}$) krzywe walidacji oscylują (epoki: $2,33 \rightarrow 2,23 \rightarrow 2,29 \rightarrow 2,23 \rightarrow 2,23$), zamiast monotonicznie maleć. Eksperymentalnie potwierdza to obserwację, że dla AdamW na zadaniach językowych 10^{-3} jest empirycznie korzystnym punktem pracy.

Sztuczne skracanie trasy przy dużym ρ . B4_reppen_1.3 wskazuje, że wysoka kara za powtórzenia może wymusić przedwczesne pojawienie się `<EOS>` (trasa 6 chwytów zamiast typowych 8–12). Token `<EOS>` nie jest objęty karą, więc jego względne prawdopodobieństwo rośnie, gdy penalizowane są wszystkie wcześniej wygenerowane chwyt.

5.3 Ograniczenia zastosowanej metodyki

- **Brak walidacji geometrycznej.** Model reprezentuje otwory wyłącznie jako abstrakcyjne tokeny i nie korzysta z ich współrzędnych (x, y) na panelu. Pozytywna obserwacja monotoniczności jest konsekwencją numerowania otworów, a nie eksplicytnego modelowania geometrii.
- **Brak kontroli długości.** Liczba chwytów w trasie jest pochodną parametrów dekodowania (zwłaszcza ρ i τ), nie zaś bezpośrednio sterowalnym warunkiem.
- **Pojedynczy holdout walidacyjny.** Pomiar `val_nll` zależy od konkretnego splitu (sec. 3.5). Stratyfikowana k -fold CV (np. po `grade_id`) dałaby precyzyjniejszą estymację.
- **Ograniczony budżet treningowy w grupie A.** 5 epok na 20 000 próbek służy jedynie przedstawieniu *trendów*; bezwzględne wartości `best_val_nll` byłyby niższe przy 50 epokach na pełnym korpusie (porównaj `report_full`: 0,858).
- **Pojedynczy prompt w grupie B.** Wszystkie testy generacji wykonano na parze (1272, 1240). Obserwacje dotyczące parametrów dekodowania pozostają jakościowo prawdziwe, ale konkretne długości tras n_h należy traktować jako orientacyjne.

- **NLL jako wskaźnik zastępczy.** Niska wartość `val_nll` nie jest jednoznaczna z wysoką jakością trasy z punktu widzenia wspinacza — wymaga oceny eksperckiej lub dodatkowych metryk geometrycznych (długość kroków, rozproszenie chwytów).

6 Wnioski

Optymalna konfiguracja uczenia. Na podstawie tabeli 1 i krzywych z rysunków 6–12 formułujemy następujący zestaw rekomendacji praktycznych dla zadania Kilter (zadanie modelowania krótkich sekwencji warunkowanych dwoma tokenami):

- **Optymalizator:** AdamW z $\text{lr} = 10^{-3}$, $\beta_1 = 0,9$, $\beta_2 = 0,999$. Niższe `lr` powoduje zauważalne niedouczenie, wyższe — niestabilność.
- **Pojemność:** `embed_dim`= 128 i `hidden_dim`= 256 stanowią uzasadniony kompromis między jakością a kosztem. Przejście na model duży (`e`=256, `h`=512) jest uzasadnione dopiero przy odpowiednio większym budżecie obliczeniowym.
- **Liczba warstw GRU:** 1–3. Pięć warstw nie poprawia wyników, a jednocześnie spowalnia zbieżność. Przy krótkich sekwencjach głębokie rekurencyjne dekodery są przeparametryzowane.
- **Dropout:** 0,0–0,15 dla małych podzbiorów, 0,15 dla pełnego korpusu. Dropout 0,5 stanowi zbyt silną regularyzację.
- **Rozmiar batcha:** małe batche (16–64) działają lepiej niż duże (256). Mniejszy batch oznacza więcej kroków optymalizatora i bardziej stochastyczny gradient, co empirycznie poprawia generalizację na małym budżecie epok.

Optymalna konfiguracja dekodowania. Z grupy B wynika, że stabilnym domyślnym profilem do generacji użytecznych tras jest:

$$\tau \in [0,7; 1,0], \quad k \in [10; 40], \quad p \in [0,9; 0,95], \quad \rho \in [1,0; 1,1].$$

Profile bardziej eksploracyjne (wyższe τ , k , p) są dopuszczalne, ale wymagają postprocesingu odrzucającego wygenerowane trasy bez ról lub o nadmiernej długości.

Zachowania emergentne. Najistotniejszym jakościowym wnioskiem jest spontaniczne pojawienie się *gramatyki przeplotu* chwyt/rola oraz porządku monotonicznego pozycji. Modele rekurencyjne z bardzo małą liczbą parametrów (`A2_small`: $\sim 0,1\text{M}$ parametrów) są w stanie uchwycić te regularności, co potwierdza, że indukcyjny bias GRU jest zgodny ze strukturą danych Kilter.

Limit obecnej architektury. Mimo poprawnej powierzchniowej struktury, model nie uwzględnia jawnie *geometrii panelu* ani *biomechaniki ruchu*. Najczęstsze błędy generacji (degeneracja przy wysokim τ , nieuzasadnione skracanie przy wysokim ρ) wskazują, że zwiększanie różnorodności wymaga uzupełnienia o moduł postprocesujący lub o jawne warunkowanie geometryczne.

7 Dalsze kierunki rozwoju

Geometryczne warunkowanie modelu. Najistotniejszym kierunkiem jest dostarczenie modelowi informacji o pozycjach chwytów na panelu. Każdy otwór ma znaną parę współrzędnych (x_i, y_i) . Informację tę można uwzględnić przez:

- statyczne wektory pozycyjne dodawane do `token_embedding`,
- wprowadzenie kontroli metrycznych odstępów między kolejnymi chwytami,
- warunkową funkcję kosztu karzącą fizycznie nierealne ruchy.

Architektury z mechanizmem uwagi. Zastąpienie GRU dekodерem typu Transformer (Sutskever et al., 2014) z self-attention pozwoli modelować zależności pomiędzy odległymi chwytami bez kompresowania całej historii do jednego stanu ukrytego. Po wprowadzeniu `condition_every_step` kontekst (`layout`, `grade`) jest już dostępny w każdym kroku GRU, jednak mechanizm uwagi nadal mógłby lepiej obsłużyć globalne zależności geometryczne i powtarzalne motywy ruchowe.

Zaawansowane strategie dekodowania.

- *Beam search* z karą za długość — alternatywa dla greedy gwarantująca stabilność.
- *Contrastive decoding* — generowanie z dwóch modeli (większego i mniejszego), zwiększające jakość.
- *Reranker* — generowanie N kandydatów i wybór najlepszego według metryki geometrycznej.

Walidacja krzyżowa i hyperparameter search. W rozszerzonej fazie projektu warto zastosować:

- stratyfikowaną k -fold CV po `grade_id` (sekcja 3.5),
- Bayesian/random search nad siatką `{lr, hidden_dim, layers, dropout, batch_size}` z biblioteką typu Optuna,
- early stopping z `patience = 3–5` oraz schedulerem typu `ReduceLROnPlateau`.

Sterowanie długością i stylem trasy. Wprowadzenie dodatkowych tokenów warunkujących, np.:

- `<LEN=k>` — pożądana liczba chwyków,
- `<STYLE=slab|overhang|dyno>` — styl trasy,

pozwoli użytkownikowi dokładniej kontrolować generowany problem bez konieczności manipulowania parametrami dekodowania.

Ocena ekspercka. Pomiar `val_nll` jest niezbędny, ale niewystarczający. Docelowo proponuje się zebranie pary „trasa wygenerowana / trasa eksperta” i ocenę przez wspina-czy (np. w skali Likerta). To umożliwiłoby dostrojenie modelu metodą *preference learning* (np. DPO), nawiązując do współczesnych technik RLHF stosowanych w modelach językowych.

Literatura

- Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Janvin. A neural probabilistic language model. *Journal of Machine Learning Research*, 3:1137–1155, 2003.
- Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014.
- Junyoung Chung, Caglar Gulcehre, KyungHyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv preprint arXiv:1412.3555*, 2014.
- Angela Fan, Mike Lewis, and Yann Dauphin. Hierarchical neural story generation. *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2018.
- Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. *International Conference on Learning Representations (ICLR)*, 2020.

- Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*, 2015.
- Ron Kohavi. A study of cross-validation and bootstrap for accuracy estimation and model selection. *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)*, 1995.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *International Conference on Learning Representations (ICLR)*, 2019.
- Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. *International Conference on Machine Learning (ICML)*, 2013.
- David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.
- stfamod. Kilter board dataset. <https://huggingface.co/datasets/stfamod/Kilter-Board-Dataset/tree/main>, 2024. dostep: 2026.
- Mervyn Stone. Cross-validatory choice and assessment of statistical predictions. *Journal of the Royal Statistical Society. Series B (Methodological)*, 36(2):111–147, 1974.
- Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. Sequence to sequence learning with neural networks. *Advances in Neural Information Processing Systems*, 27, 2014.
- Ronald J. Williams and David Zipser. A learning algorithm for continually running fully recurrent neural networks. *Neural Computation*, 1(2):270–280, 1989.